DevOps Engineer Case Study

Implementation Report: Jenkins CI/CD, Kubernetes and Pipeline Optimization

Scope

Mandatory Task 0 (Kubernetes), plus Task 1 (Jenkins CI/CD) and Task 4 (Pipeline Optimization). This is a starter implementation and does not claim that deployment has already been executed.

Architecture

GitHub hosts source, Jenkinsfile and manifests. Jenkins Controller orchestrates jobs; a dedicated Linux agent runs tests and Docker build. The pipeline publishes commit-tagged images to Docker Hub and updates Kubernetes on main.

Pipeline

Stages: checkout; unit tests; Docker build; push on main using Jenkins credential binding; deploy on main using a restricted kubeconfig; wait for rollout status. Pipeline timeout and concurrent-build control reduce runaway or overlapping deployments.

Kubernetes decisions

kubectl apply is used because this demo has only Namespace, Deployment and Service. Helm is a reasonable next step for reusable charts and many environment values. Deployment uses two replicas, RollingUpdate (maxUnavailable=0, maxSurge=1), health probes, resource requests/limits and a non-root container.

Rollback and scale

Run kubectl rollout undo deployment/demo-web -n devops-demo and verify rollout status. Scale manually with kubectl scale deployment/demo-web --replicas=3 -n devops-demo. HPA would require metrics-server and resource requests; it is not enabled in this starter. Database migrations need a separate backward-compatible strategy.

Task 4 optimization

For a ~25-minute build, first measure stage durations and classify failures. Then: cache dependencies; optimize Docker layer ordering; parallelize independent tests/scans when capacity allows; build once and promote the same immutable image; right-size or autoscale agents.

Monitoring and metrics

Track Jenkins queue depth, agent availability, stage duration and build success rate. Track app latency, request/error rate, Pod restarts and CPU/memory. Leadership metrics: deployment frequency, build success rate and MTTR. Alert on failed rollouts, repeated build failures and sustained latency/5xx increases.

Run demo

On Linux install Docker Compose, Python 3.12, kubectl and kind. Run docker compose up --build -d app and curl localhost:8081/health. Run tests with python -m unittest discover -s app/tests -v. Start kind, replace the Docker Hub placeholder in k8s/deployment.yaml, apply manifests and port-forward the Service.

Troubleshooting

ImagePullBackOff: check image tag, visibility and pull secret. CrashLoopBackOff: inspect logs, describe Pod, environment and probes. Rollout timeout: inspect events, readiness and resources. Push denied: check token and credential ID. Agent offline: check network, node name, secret and label.

Security and limitations

Never commit secrets; use Docker Hub access token and namespace-scoped kubeconfig. A build agent needs Docker CLI/daemon, Python, kubectl and Git. Avoid exposing Docker socket to untrusted jobs. Before submission, run all steps, capture real evidence, and replace placeholders. Production should add scanning/signing, approvals, observability, RBAC and secret management.